# MODULE-6

MANISH SHARMA

INTRODUCTION TO PYTHON

THEORY:

## • Introduction to Python and its Features (simple, high-level, interpreted language)

Python is a high-level, interpreted, and general-purpose programming language, known for its simplicity, readability, and versatility. It was created by Guido van Rossum and first released in 1991. Python has since become one of the most popular programming languages, used in various fields, such as web development, data science, artificial intelligence, machine learning, automation, and more.

Features of Python Easy to Learn and Use: Python's syntax is clear and closely resembles the English language, making it beginner-friendly.

Interpreted Language: Python code is executed line by line, which simplifies debugging and allows for quick testing of code snippets.

Dynamically Typed: Variables in Python do not require explicit type declarations. The type of a variable is determined at runtime.

Object-Oriented: Python supports object-oriented programming (OOP), enabling developers to create reusable code through classes and objects.

Extensive Standard Library: Python has a rich set of built-in libraries that support tasks such as file I/O, regular expressions, and networking, among others.

Cross-platform: Python is platform-independent, meaning it can run on various operating systems, such as Windows, macOS, and Linux.

Open Source: Python is free to use, distribute, and modify, which has led to a large and active community of developers contributing to its growth.

Extensibility: Python can integrate with other programming languages, such as C, C++, and Java, to optimize performance-critical sections of code.

## • History and evolution of Python

Python is one of the most popular programming languages in the world today. It was created to provide a simple, readable, and powerful programming language for developers.

# MODULE-6

MANISH SHARMA

## Origin of Python

Python was developed by Guido van Rossum in the late 1980s at the Centrum Wiskunde & Informatica (CWI) in the Netherlands.

- Work on Python started in December 1989
- The first public release, Python 0.9.0, came in 1991

The name “Python” was inspired by the British comedy show:

- Monty Python's Flying Circus

and not from the snake.

## Goals Behind Python

Guido van Rossum designed Python with the following goals:

- Simple and easy-to-read syntax
- Reduce complex coding
- Improve programmer productivity
- Support code reusability
- Provide portability across systems

## Evolution of Python

### 1. Python 0.9.0 (1991)

The first version included:

- Classes and inheritance
- Functions
- Exception handling
- Core data types like:
    - List
    - Dictionary
    - String

This version already supported many modern programming concepts.

### 2. Python 1.0 (1994)

Python 1.0 introduced:

- Functional programming tools
    - lambda
    - map()
    - filter()
    - reduce()

It became more stable and started gaining popularity among programmers.

### 3. Python 2.0 (2000)

Released by the Python Software Foundation.

Major features added:

- List comprehensions
- Garbage collection system
- Unicode support

Example of list comprehension:

```
squares = [x*x for x in range(5)]
print(squares)
```

Python 2 became widely used in industry and education.

### End of Python 2

Support for Python 2 officially ended on **January 1, 2020**.

### 4. Python 3.0 (2008)

Python 3.0 was a major redesign of the language.

Key improvements:

- Better Unicode handling

- Improved syntax consistency
- Cleaner and more efficient code structure

Example:

print("Hello")

Python 3 removed many older features from Python 2 to make the language simpler and more modern.

### 5. Modern Python (Python 3.x)

Current Python versions continue to improve:

- Performance
- Security
- Libraries
- Developer tools

Modern Python supports advanced technologies such as:

- Artificial Intelligence
- Machine Learning
- Data Science
- Cloud Computing
- Automation

Popular frameworks and libraries:

- Django
- Flask
- TensorFlow
- PyTorch

# MODULE-6

MANISH SHARMA

## • Advantages of using Python over other programming languages.

**1. Easy to Learn and Use**

● **Readable Syntax:** Python's syntax is simple and similar to English, which makes it easy to read and understand. This is especially beneficial for beginners.

● **Minimalist Syntax**: Python requires fewer lines of code compared to many other languages, making it easier to write and maintain.

**2. Cross-Platform Compatibility**

● **Python is platform-independent,** which means code written in Python can run on any operating system (Windows, Linux, macOS) without modification

**3. Extensive Libraries and Frameworks**

● Python offers a wide range of pre-built libraries and frameworks for various tasks, including:

○ Web development**:** Django, Flask ○ Data Science & Machine Learning: Pandas, NumPy, TensorFlow, Scikit-learn

GUI development: Tkinter, PyQt ○ Automation: Selenium, OpenPyXL

**4. Large Community Support**

● Python has a vast and active community that provides support through forums, tutorials, and open-source contributions. This makes it easier to find solutions to problems and learn from others.

**5. Integration Capabilities**

● Python integrates easily with other programming languages like C, C++, and Java.

● It also supports integration with technologies such as REST APIs and databases.

**6. Versatile Applications**

● Python can be used for a wide range of applications, including: ○ Web development ○ Data analysis and visualization ○ Machine learning and artificial intelligence ○ Scripting and automation ○ Game development

**7. Great for Prototyping and Development Speed**

# MODULE-6

## MANISH SHARMA

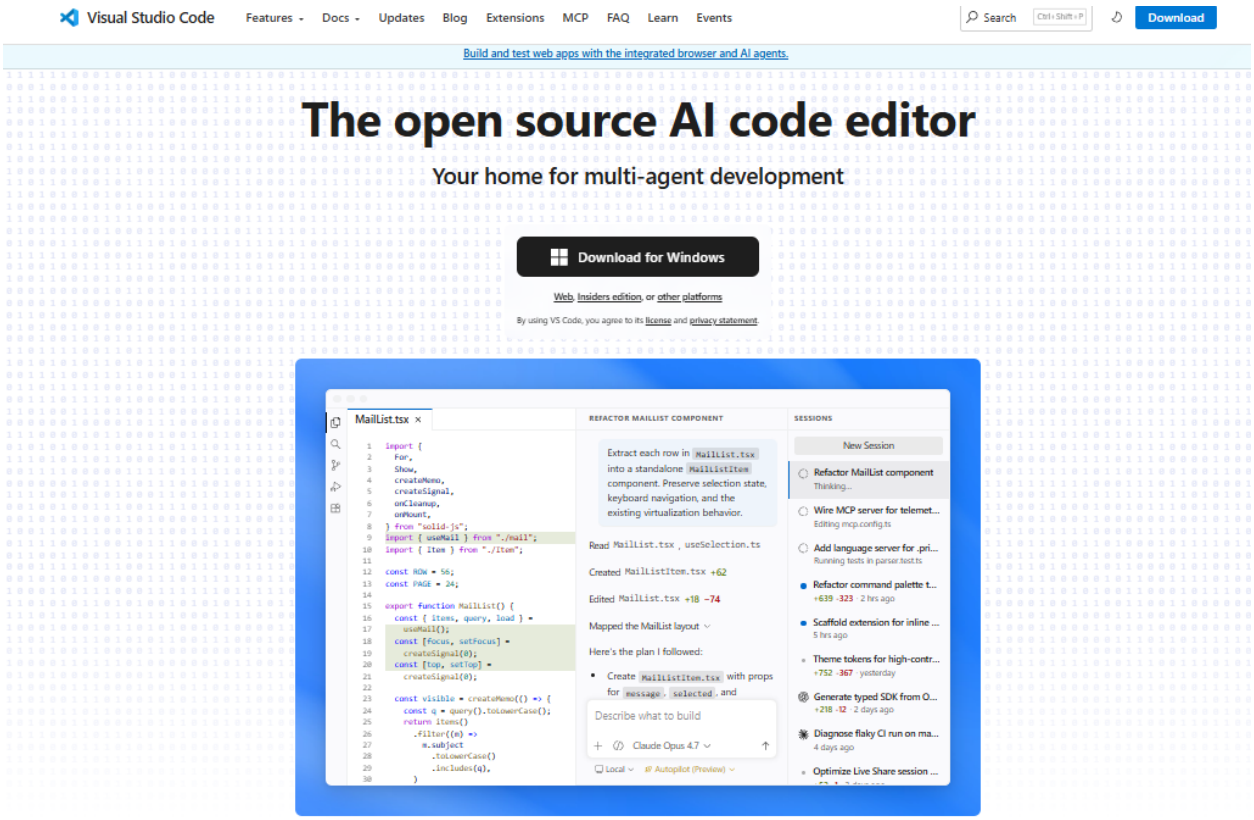
● Python is known for its rapid prototyping capabilities, which allows developers to create and test applications faster.

**• Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).**

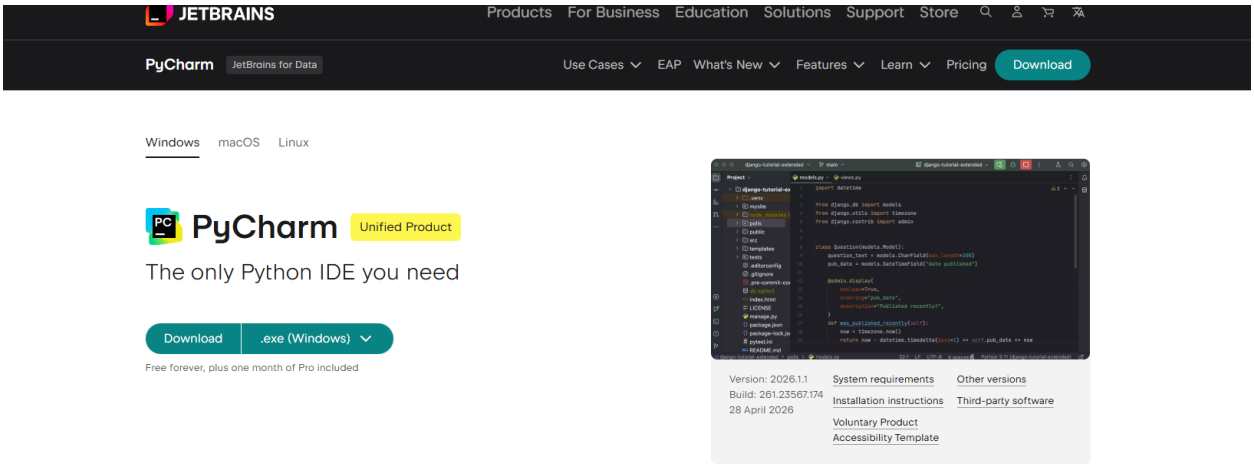

https://www.python.org/downloads/?utm_source=chatgpt.com

# MODULE-6

## MANISH SHARMA



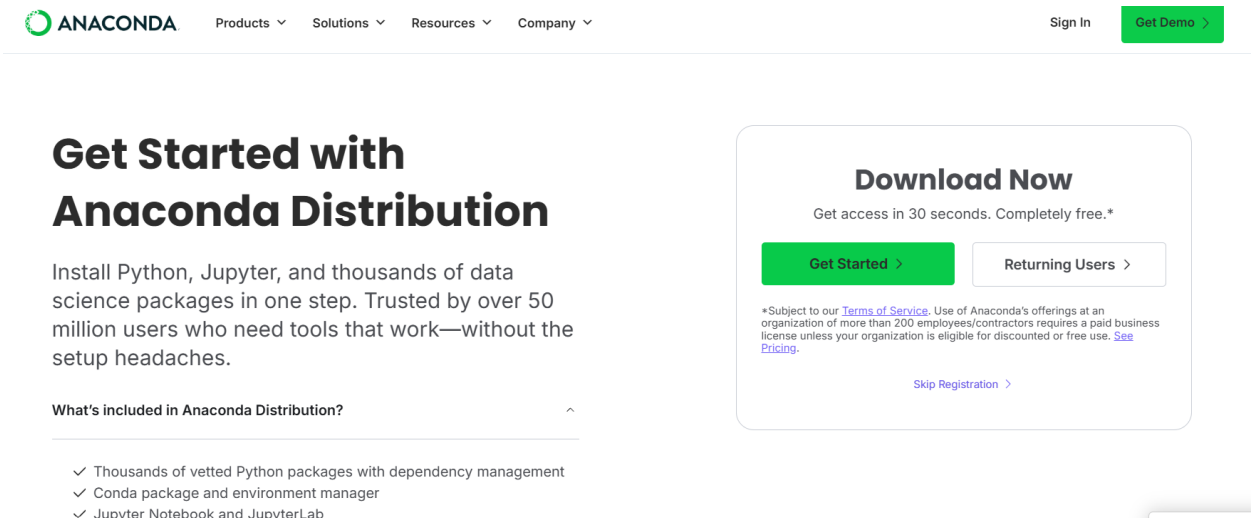

https://code.visualstudio.com/




https://www.jetbrains.com/pycharm/download/?utm_source=chatgpt.com

# MODULE-6

MANISH SHARMA




https://www.anaconda.com/download?utm_source=chatgpt.com

- **Writing and executing your first Python program.**

Print("hello tops")

## 2. PROGRAMMING STYLE

### THEORY:

- **Understanding Python's PEP 8 guidelines**.

Understanding PEP 8 Guidelines

What is PEP 8?

PEP 8 stands for:

Python Enhancement Proposal 8

It is the official style guide for writing code in Python.

PEP 8 was created to improve:

- Code readability
- Consistency
- Maintainability

# MODULE-6

## MANISH SHARMA

It provides rules and recommendations on how Python code should be written and formatted.

Official documentation:

### Why is PEP 8 Important?

Following PEP 8 helps programmers:

- Write clean code
- Make code easier to understand
- Improve teamwork
- Reduce errors
- Maintain professional coding standards

### Main PEP 8 Guidelines

#### 1. Indentation

Use **4 spaces per indentation level**.

Correct:

```
if True:
    print("Hello")
```

Incorrect:

```
if True:
  print("Hello")
```

#### 2. Maximum Line Length

- Recommended maximum line length:
    - **79 characters**

Long lines should be split into multiple lines.

Correct:

```
message = (
    "Python is a simple and powerful programming language"
```

)

### 3. Naming Conventions

#### Variable and Function Names

Use:

- lowercase letters
- words separated by underscores

Example:

```
student_name = "manish"


def calculate_total():
    pass
```

#### Class Names

Use:

- CapitalizedWords (CamelCase)

Example:

```
class StudentDetails:
    pass
```

#### Constants

Use:

- UPPERCASE letters

Example:

```
PI = 3.14159
MAX_USERS = 100
```

### 4. Spaces Around Operators

Use spaces around operators and after commas.

Correct:

x = 10 + 5

Incorrect:

x=10+5

### 5. Blank Lines

Use blank lines to separate:

- Functions
- Classes
- Logical sections of code

Example:

```
def add():
    pass



def subtract():
    pass
```

### 6. Import Statements

Imports should usually be written:

- One per line
- At the top of the file

Correct:

```
import math
import random
```

Incorrect:

```
import math, random
```

# MODULE-6

# MANISH SHARMA

7. Comments and Documentation

Comments should be clear and meaningful.

Example:

# Calculate total marks

total = marks1 + marks2

8. Avoid Extra Spaces

Correct:

numbers = [1, 2, 3]

Incorrect:

numbers = [ 1, 2, 3 4 ]

- **Indentation, comments, and naming conventions in Python**

1-Indentation:

Age=18

If age >=18 :

Print(“eligible to vote”)

2- comments:

#addision

A=18

B=02

Sum=A+B

Print(Sum)

3- naming conventions in python:

name = "manish"

marks=80

print(f"my name is {name} and my marks is {marks}")

# MODULE-6

## MANISH SHARMA

- **Writing readable and maintainable code**

#addision

A=18

B = 02

Sum = A + B

Print(Sum)

name = "manish"

marks=80

print(f"my name is {name} and my marks is {marks}")

### 3. CORE PYTHON CONCEPTS

### THEORY:

- **Understanding data types: integers, floats,strings, lists, tuples, dictionaries,sets**

Understanding Data Types in Python

What are Data Types?

Data types define the type of data a variable can store in a program.

Python provides several built-in data types, including:

- Integers
- Floats
- Strings
- Lists
- Tuples
- Dictionaries
- Sets

---

# MODULE-6

## MANISH SHARMA

1. Integer (int)

Definition

Integers are whole numbers without decimal points.

Examples:

- Positive numbers
- Negative numbers
- Zero

Example

age = 25

temperature = -5

count = 0

print(age)

print(temperature)

print(count)

Output

25

-5

0

2. Float (float)

Definition

Floats are numbers containing decimal points.

Examples:

- 3.14
- 10.5

- -7.25

Example

price = 99.99

height = 5.8

print(price)

print(height)

Output

99.99

5.8

3. String (str)

Definition

Strings are sequences of characters enclosed in:

- Single quotes ' '
- Double quotes " "

Used to store text data.

Example

name = "Rahul"

message = 'Welcome to Python'

print(name)

print(message)

Output

Rahul

Welcome to Python

# MODULE-6

## MANISH SHARMA

4. List (list)

Definition

A list is an ordered and mutable collection of items.

Features:

- Allows duplicate values
- Items can be changed
- Written using square brackets [ ]

Example

numbers = [10, 20, 30, 40]

print(numbers)

print(numbers[1])

Output

[10, 20, 30, 40]

20

Modifying a List

numbers[2] = 100

print(numbers)

Output

[10, 20, 100, 40]

5. Tuple (tuple)

Definition

A tuple is an ordered but immutable collection of items.

Features:

- Cannot be modified after creation
- Allows duplicate values
- Written using parentheses ( )

Example

```
colors = ("red", "green", "blue")


print(colors)
print(colors[0])
```

Output

```
('red', 'green', 'blue')
red
```

6. Dictionary (dict)

Definition

A dictionary stores data in:

- Key-value pairs

Features:

- Mutable
- Unordered (in older versions)
- Written using curly braces { }

Example

```
student = {
   "name": "Amit",
   "age": 20,
   "marks": 85
}
```

print(student)

print(student["name"])

Output

{'name': 'Amit', 'age': 20, 'marks': 85}

Amit

7. Set (set)

Definition

A set is an unordered collection of unique items.

Features:

- No duplicate values
- Mutable
- Written using curly braces { }

Example

numbers = {1, 2, 3, 4, 4, 5}

print(numbers)

Output

{1, 2, 3, 4, 5}

Duplicate values are automatically removed.

## • Python variables and memory allocation.

Variable :

Variable is a name which is contain some value. Variable Rules:

Variable Names Must Begin with a Letter or Underscore:

# MODULE-6

## MANISH SHARMA

The first character of a variable name must be a letter (a-z, A-Z) or an underscore (_). For example, age, name, _value, and score1 are valid variable names.

Variable Names Can Contain Letters, Numbers, and Underscores:

After the first character, the variable name can include letters (a-z, A-Z), digits (0-9), and underscores (_). For example, age_1, student_score, and value_2 are valid variable names.

Variable Names Are Case-Sensitive:

o Python treats uppercase and lowercase letters as different. For example, age and Age are two different variables.

o age, Age, and AGE are considered distinct. Variable Names Cannot Be Reserved Keywords:

o Python has a set of reserved keywords (like class, for, if, True, False, def, etc.) that cannot be used as variable names.

o For example, if, for, and while are not allowed as variable names. Variable Names Should Be Meaningful

o While not a strict rule, it's good practice to choose meaningful names that convey the purpose of the variable. For example, age for a person's age, total_amount for a total cost, etc.

### Memory Management

Python uses an automatic memory management system that involves dynamic typing, reference counting, and garbage collection. These components work together to efficiently allocate and deallocate memory during the program's execution. Heap Memory: Python objects (like integers, strings, lists, etc.) are stored in the heap (dynamic memory). Stack Memory: Local variables (such as function arguments) and function calls are managed in the stack.

- **Python operators: arithmetic, comparison, logical, bitwise.**

# MODULE-6

## MANISH SHARMA

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

| Operator | Meaning | Example |
|---|---|---|
| + | Addition | 5 + 2 = 7 |
| - | Subtraction | 5 - 2 = 3 |
| * | Multiplication | 5 * 2 = 10 |
| / | Division | 5 / 2 = 2.5 |
| % | Modulus (remainder) | 5 % 2 = 1 |
| ** | Exponent | 5 ** 2 = 25 |
| // | Floor division | 5 // 2 = 2 |

2. Comparison Operators

| Operator | Meaning | Example |
|---|---|---|
| == | Equal to | `5 == 5 → True` |
| != | Not equal to | `5 != 3 → True` |
| > | Greater than | `5 > 3 → True` |
| < | Less than | `5 < 3 → False` |
| >= | Greater or equal | `5 >= 5 → True` |
| <= | Less or equal | `5 <= 3 → False` |

# MODULE-6

## MANISH SHARMA

Logical Operators in Python

| Operator | Name | Example | Result |
|---|---|---|---|
| and | Logical AND | True and False | False |
| or | Logical OR | True or False | True |
| not | Logical NOT | not True | False |

### 4. CONDITIONAL STATEMENTS

### THEORY:

- **Introduction to conditionalstatements: if, else, elif**

Example:-

```
age = 18


if age >= 18:
    print("You are eligible to vote")
```

- **Nested if-else conditions.**

Example:-

```
age = 20
```

```
has_id = True


if age >= 18:
    if has_id:
        print("Allowed to enter")
    else:
        print("ID required")
else:
    print("Not eligible")
```

## 5. LOOPING (FOR, WHILE)

### THEORY:

### Introduction to for and while loops.

1. for Loop

Definition

A for loop is used when we know how many times we want to repeat a block of code.

It is commonly used to iterate over:

- Lists
- Strings
- Ranges of numbers

2-loop

- while Loop
- Definition
- A while loop is used when we don't know the exact number of iterations. It runs as long as the condition is True.

# MODULE-6

## MANISH SHARMA

### How loops work in Python

(a)For loop:-

```
for i in range(1, 4):
    print(i)
```

(b)while loop:-

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

### Using loops with collections (lists, tuples, etc.).

List-

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
```

tuple-

```
  print(fruit)
colors = ("red", "green", "blue")

for color in colors:
    print(color)
```

### 8. CONTROL STATEMENTS (BREAK, CONTINUE, PASS)

### THEORY:

# MODULE-6

## MANISH SHARMA

- **Understanding the role of break, continue, and pass in Python loops**

| Statement | Purpose | Effect |
|---|---|---|
| break | Exit loop | Stops completely |
| continue | Skip iteration | Moves to next loop cycle |
| pass | Do nothing | Placeholder |

### 9. STRING MANIPULATION

### THEORY:

- **Understanding how to access and manipulate strings.**

Understanding How to Access and Manipulate Strings in Python

What is a String?

A string is a sequence of characters enclosed in:

- Single quotes ' '
- Double quotes " "

Example:

text = "Python"

---

1. Accessing Strings (Indexing)

Each character in a string has an index starting from 0.

P y t h o n

0 1 2 3 4 5

---

Example

text = "Python"

```
print(text[0])  # First character
print(text[3])  # Fourth character
```

Output:

```
P
h
```

---

Negative Indexing

Python also allows accessing from the end:

```
 P  y  t  h  o  n
-6 -5 -4 -3 -2 -1
```

```
text = "Python"


print(text[-1])
print(text[-2])
```

Output:

```
n
o
```

---

2. Slicing Strings

Slicing means extracting part of a string.

Syntax:

```
string[start:end]
```

- Start index is included
- End index is not included

---

Example

```
text = "Python"


print(text[0:3])
```

Output:

```
Pyt
```

---

More Examples

```
text = "Python"


print(text[2:6])

print(text[:4])

print(text[3:])
```

---

3. String Manipulation Methods

Python provides many built-in methods to modify strings.

---

1. Uppercase

```
text = "python"

print(text.upper())
```

Output:

```
PYTHON
```

---

2. Lowercase

```
text = "PYTHON"

print(text.lower())
```

Output:

```
python
```

---

3. Replace

```
text = "I like Java"

print(text.replace("Java", "Python"))
```

Output:

```
I like Python
```

---

4. Length of String

```
text = "Python"

print(len(text))
```

Output:

```
6
```

---

5. Strip (remove spaces)

```
text = "  Python  "

print(text.strip())
```

Output:

```
Python
```

---

4. Looping Through a String

You can access each character using a loop:

```
text = "Python"

for char in text:
    print(char)
```

---

5. Checking Substring

```
text = "I love Python"

if "Python" in text:
    print("Found")
```

Output:

Found

- **Basic operations: concatenation, repetition,string methods(upper(), lower(), etc.).**

| Operation | Symbol / Method | Example |
|---|---|---|
| Concatenation | + | "A" + "B" |
| Repetition | * | "A" * 3 |
| Uppercase | upper() | "a".upper() |
| Lowercase | lower() | "A".lower() |
| Capitalize | capitalize() | "hello".capitalize() |
| Length | len() | len("abc") |

- **String slicing**

String Slicing in Python

# MODULE-6

## MANISH SHARMA

| Operation | Symbol / Method | Example |
|---|---|---|

What is String Slicing?

String slicing is the process of extracting a part (substring) from a string using its index positions.

A string is a sequence of characters, and each character has an index starting from 0.

Example:

text = "Python"

# Index: 0 1 2 3 4 5

#  P y t h o n

---

1. Basic Syntax of Slicing

string[start : end]

Rules:

- start → included
- end → not included

---

2. Simple Example

text = "Python"

print(text[0:3])

Output:

Pyt

---

| Operation | Symbol / Method | Example |
|---|---|---|

3. Omitting Start or End

a) Without start index

```
text = "Python"


print(text[:4])
```

Output:

Pyth

---

b) Without end index

```
text = "Python"


print(text[2:])
```

Output:

thon

---

4. Negative Index Slicing

Negative indexing starts from the end of the string.

P y t h o n

-6 -5 -4 -3 -2 -1

---

Example

```
text = "Python"
```

| Operation | Symbol / Method | Example |
|---|---|---|

print(text[-4:-1])

Output:

tho

---

5. Step Value in Slicing

Syntax:

string[start:end:step]

- Step defines skipping characters

---

Example

text = "Python"

print(text[0:6:2])

Output:

Pto

---

6. Reverse a String Using Slicing

text = "Python"

print(text[::-1])

Output:

nohtyP

| Operation | Symbol / Method | Example |
|---|---|---|

7. Real-Life Example

email = "student@example.com"

username = email[:7]

print(username)

Output:

student

Summary Table

| Slicing Type | Example | Result |
|---|---|---|
| Basic | text[0:3] | First 3 characters |